

› Question 1: Loan Eligibility Calculator (Functions)

↳ 2 cells hidden

› Question 2: Function Parameters - Flexible Calculator

↳ 4 cells hidden

› Question 3: Nested Functions - Invoice Calculator

↳ 3 cells hidden

› Question 4: Functions Returning Functions - Currency Converter Factory

↳ 2 cells hidden

› Question 5: Lambda Functions - Data Processing

↳ 2 cells hidden

› Question 6: Decorators - Function Enhancement

↳ 2 cells hidden

✓ Question 7: Custom Module Creation

Description

Create a custom module with utility functions and use it in a main program.

Requirements

Create a module `math_utils.py` with:

- `calculate_percentage(part, whole)`: Calculate percentage
- `calculate_average(numbers)`: Calculate average
- `find_max_min(numbers)`: Find maximum and minimum
- `is_prime(n)`: Check if number is prime

Create a module `string_utils.py` with:

- `reverse_string(text)`: Reverse a string
- `count_words(text)`: Count words in text
- `capitalize_words(text)`: Capitalize each word
- `remove_duplicates(text)`: Remove duplicate characters

Create `main.py` that:

- Imports functions from both modules
- Uses different import methods (`import`, `from...import`, `alias`)
- Demonstrates usage of all functions

File Structure

project/

■■■ `math_utils.py`

■■■ `string_utils.py`

■■■ main.py

Input/Output Example

=== Math Utilities ===

Percentage: 75.0%

Average: 85.5

Max: 100, Min: 70

Is 17 prime? True

=== String Utilities ===

Reversed: !dlroW olleH

Word Count: 3

Capitalized: Hello World Python

No Duplicates: Helo Wrđ

Constraints

- Add docstrings to all functions
- Handle edge cases (empty lists, None values)
- Use proper module structure

```
%%writefile math_utils.py

"""
math_utils.py
"""

def calculate_percentage(part, whole):
    if whole == 0:
        return 0
    return (part / whole) * 100

def calculate_average(numbers):
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)

def find_max_min(numbers):
    if not numbers:
        return None, None
    return max(numbers), min(numbers)

def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False

    return True
```

Writing math_utils.py

```
%%writefile string_utils.py

"""
string_utils.py
"""

def reverse_string(text):
    return text[::-1]

def count_words(text):
    return len(text.split())

def capitalize_words(text):
    return text.title()
```

```
def remove_duplicates(text):
    seen = set()
    result = ""

    for ch in text:
        if ch not in seen:
            result += ch
            seen.add(ch)

    return result
```

Writing string_utils.py

```
# main.py

import math_utils
from string_utils import reverse_string
import string_utils as su

print("=== Math Utilities ===")

percentage = math_utils.calculate_percentage(
    75,
    100
)

print(f"Percentage: {percentage}%")

numbers = [70, 80, 92, 100]

average = math_utils.calculate_average(numbers)

print(f"Average: {average}")

maximum, minimum = math_utils.find_max_min(numbers)

print(f"Max: {maximum}, Min: {minimum}")

print(
    f"Is 17 prime? "
    f"{math_utils.is_prime(17)}"
)

print("\n=== String Utilities ===")

text = "Hello World!"

print(
    f"Reversed: "
    f"{reverse_string(text)}"
)

sentence = "hello world python"

print(
    f"Word Count: "
    f"{su.count_words(sentence)}"
)

print(
    f"Capitalized: "
    f"{su.capitalize_words(sentence)}"
)

print(
    f"No Duplicates: "
    f"{su.remove_duplicates(text)}"
)
```

```
=== Math Utilities ===
Percentage: 75.0%
Average: 85.5
Max: 100, Min: 70
Is 17 prime? True

=== String Utilities ===
Reversed: !dlroW olleH
```

Word Count: 3
Capitalized: Hello World Python
No Duplicates: Helo Wrld!

Start coding or [generate](#) with AI.

Question 8: Package Creation and Organization

Description

Create a package structure with multiple modules and demonstrate package usage.

Requirements

Create a package calculator with the following structure:

calculator/ init .py basic.py (basic operations) advanced.py (advanced operations) scientific.py (scientific operations)

init .py should:

- Import commonly used functions
- Define package version
- Export main functions

Each module should contain related functions:

- basic.py: add, subtract, multiply, divide
- advanced.py: power, sqrt, factorial
- scientific.py: sin, cos, log

Create main.py that uses the package

Input/Output Example

From init .py imports

from calculator import add, multiply, power

result1 = add(10, 20) # 30

result2 = power(2, 8) # 256

Direct module imports

from calculator.advanced import factorial

result3 = factorial(5) # 120

Using package

import calculator.scientific as sci

result4 = sci.sin(3.14159/2) # ~1.0

Constraints

- Proper init .py implementation
- Clear module organization
- Document package structure

```
!mkdir -p calculator
```

```
%%writefile calculator/basic.py
```

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
def multiply(a, b):  
    return a * b
```

```
def divide(a, b):
```

```

if b == 0:
    raise ValueError("Division by zero")
return a / b

```

Writing calculator/basic.py

```
%%writefile calculator/advanced.py
```

```

import math

def power(a, b):
    return a ** b

def sqrt(x):
    return math.sqrt(x)

def factorial(n):
    return math.factorial(n)

```

Writing calculator/advanced.py

```
%%writefile calculator/scientific.py
```

```

import math

def sin(x):
    return math.sin(x)

def cos(x):
    return math.cos(x)

def log(x):
    return math.log(x)

```

Writing calculator/scientific.py

```
%%writefile calculator/__init__.py
```

```

__version__ = "1.0.0"

from .basic import add, subtract, multiply, divide
from .advanced import power, sqrt, factorial

__all__ = [
    "add",
    "subtract",
    "multiply",
    "divide",
    "power",
    "sqrt",
    "factorial"
]

```

Writing calculator/__init__.py

```
!find calculator
```

```

calculator
calculator/__init__.py
calculator/advanced.py
calculator/basic.py
calculator/scientific.py

```

```
from calculator import add, multiply, power
```

```

print(add(10, 20))
print(multiply(5, 6))
print(power(2, 8))

```

```

30
30
256

```

```
from calculator.advanced import factorial
```

```
print(factorial(5))
```

```
120
```

```
import calculator.scientific as sci
```

```
print(sci.sin(3.14159/2))
```

```
0.9999999999991198
```